Azure Security Baseline & Governance (Least-Privilege, Secure Config, Guardrails)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Azure Security Baseline & Governance (Microsoft Azure) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Azure Security Baseline & Governance 4

What Azure security baseline and governance is . . . 5

Why this matters . . . 5

How it works . . . 5

What you can control . . . 5

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Azure Security Baseline & Governance 6

2.1 An Azure subscription and tenant . . . 6

2.2 Elevated rights to assign policy and RBAC . . . 6

2.3 A governance scope design (management groups / subscriptions) . . . 7

2.4 Azure Policy available . . . 7

2.5 A secure-configuration baseline definition . . . 7

2.6 Microsoft Defender for Cloud plan decision . . . 7

2.7 Change control, pilot scope, and rollback . . . 7

Checklist . . . 8

3. Implementation Guide — Azure Security Baseline & Governance 8

Naming (set once, reuse verbatim) . . . 8

Step 1 — Define and confirm the scope . . . 8

Step 2 — Assign the guardrail initiative in audit . . . 9

Step 3 — Review the audit picture and exempt false positives . . . 9

Step 4 — Tighten RBAC to least privilege . . . 9

Step 5 — Apply the secure-configuration baseline . . . 9

Step 6 — Enable Defender for Cloud and remediate top findings . . . 9

Step 7 — Validate on the pilot . . . 10

Step 8 — Move guardrails from audit to deny (the gate) . . . 10

Step 9 — Expand to production . . . 10

5. Testing Guide — Azure Security Baseline & Governance 11

Test 1 — Guardrail initiative assigned (audit) . . . 11

Test 2 — Compliance state populated . . . 11

Test 3 — RBAC is least privilege . . . 11
Test 4 — Secure-config baseline present . . . 11
Test 5 — Defender for Cloud posture . . . 11
Test 6 — Audit does not block (audit stage) . . . 11
Test 7 — Deny blocks (after the gate) . . . 11
Test 8 — Drift report . . . 11
Pass/fail gate . . . 12
6. Troubleshooting Guide — Azure Security Baseline & Governance 12
A. Policy assignment not taking effect . . . 12
B. Deny blocks legitimate work . . . 12
C. Deny didn't clean up existing resources . . . 13
D. RBAC change broke automation or access . . . 13
E. Defender for Cloud secure score not showing / not improving . . . 13
F. Diagnostics quick reference . . . 13
G. When to escalate . . . 13
7. Best Practices — Azure Security Baseline & Governance 13
8. Deliverables Summary — Azure Security Baseline & Governance 14
Documentation . . . 14
PowerShell scripts . . . 15
What the buyer receives . . . 15
Outcome . . . 15
Appendix A — PowerShell Scripts 16

Executive Summary — Azure Security Baseline & Governance (Microsoft Azure)

White-label · Azure Solutions

An Azure subscription without governance drifts fast: over-privileged owners, public storage, unencrypted resources, and no consistent baseline — exactly what Defender for Cloud and auditors flag. This solution puts **guardrails** around Azure so secure configuration, least privilege, and a consistent baseline are the default the platform enforces, not something an admin has to remember.

This is a complete, **deploy-it-today** package for standing up an Azure security baseline and governance model, written for an administrator with no prior Azure Policy or governance experience.

What you get

- A plain-language **Solution Overview** of management groups, Azure Policy initiatives, RBAC least-privilege, the secure-config baseline, and Defender for Cloud.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite.
- An **Implementation Guide** of precise, ordered portal steps in the required order — scope, guardrails audit-then-deny, RBAC tightening, baseline, and Defender.
- Five **PowerShell scripts** for policy/RBAC/Defender discovery, compliance and least-privilege validation, a governance posture and drift report, safe remediation, and a scope-targeting helper.
- **Testing** and **Troubleshooting** guides for confident rollout.

What this delivers

Azure Policy guardrail initiatives assigned **audit-then-deny** (require encryption, block public storage, allowed locations and SKUs), **RBAC** tightened to least privilege with custom roles and no standing Owner, a **secure-configuration baseline** applied estate-wide, and **Microsoft Defender for Cloud** measuring and improving secure score and regulatory compliance — all proven on a pilot scope first.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with an audit-before-deny gate and a documented rollback.

Outcome

Azure under governance — policy guardrails preventing misconfiguration, RBAC at least privilege, a secure-config baseline applied, and Defender for Cloud posture improving — instead of drift and audit findings, deployable by a single administrator and validated before you scale.

1. Solution Overview — Azure Security Baseline & Governance

What Azure security baseline and governance is

Azure security baseline and governance is the discipline of putting **enforced guardrails** around an Azure estate so that secure configuration, least-privilege access, and a consistent baseline are the default — not something an admin has to remember. It combines four Azure capabilities: **management groups** (a hierarchy that lets one control apply to many subscriptions), **Azure Policy initiatives** (rules that audit or block non-compliant resources), **Azure role-based access control (RBAC)** tightened to least privilege, and **Microsoft Defender for Cloud** to measure and continuously improve posture.

Why this matters

An Azure subscription without governance drifts fast. Owners accumulate standing privilege, storage accounts get created with public access, resources are deployed unencrypted or in disallowed regions, and there is no consistent baseline anyone can audit against — exactly the findings Defender for Cloud and external auditors flag. Governance turns those one-off mistakes into things Azure itself prevents (deny) or at minimum records (audit), and it gives you a single secure-score number that trends the right way over time. The goal is an estate that stays secure because the platform enforces it, proven on a small pilot scope before it is applied everywhere.

How it works

1. A **management-group and subscription scope** is defined so one policy or role assignment can cover many subscriptions from a single point in the hierarchy.

2. **Azure Policy initiatives** (grouped policy sets) are assigned at that scope in **audit** effect first — they report non-compliant resources without blocking anything.

3. Once the audit picture is understood and false positives are exempted, the same guardrails are moved to **deny** effect so non-compliant resources can no longer be created.

4. **RBAC is tightened to least privilege** — standing Owner assignments are reduced, purpose-built **custom roles** replace over-broad built-in roles, and access is granted at the narrowest scope that still lets people work.

5. A **secure-configuration baseline** (encryption required, public network access restricted, allowed locations and SKUs, secure transfer, diagnostic logging) is expressed as policy so every resource inherits it.

6. **Microsoft Defender for Cloud** is enabled to produce a **secure score**, a **regulatory compliance** view, and prioritized **recommendations**; the top findings are remediated and the score is tracked each cycle.

What you can control

• **Guardrail effect** — audit (report only) or deny (block creation) per policy, moved from one to the other on a controlled gate.

• **Least-privilege access** — custom roles, removal of standing Owner, and scope-narrowed assignments.

• **Allowed configuration** — required encryption, restricted public storage/network access, allowed locations (regions) and allowed SKUs/resource types.

• **Posture measurement** — Defender for Cloud secure score, regulatory-compliance mapping, and recommendation remediation.

• **Scope** — which management groups and subscriptions each guardrail and role applies to.

Limitations (be honest with buyers)

• A **deny** policy blocks *new* non-compliant resources; it does not delete or reconfigure resources that already exist — those are remediated separately (audit findings, remediation tasks, or the remediation script).

• Some Defender for Cloud plans and features are **paid** per resource type; the free foundational CSPM covers secure score and recommendations, enhanced plans add more.

• Policy evaluation and compliance state are **not instantaneous** — new assignments can take time to evaluate; a manual compliance scan speeds this up.

• Tightening RBAC can break automation or people who relied on standing privilege — this is exactly why the model pilots on a narrow scope first and keeps a rollback.

• Governance reduces risk; it does not replace patching, backup, or workload-specific security controls.

The standard deployment model

> ***Define scope (management groups/subscriptions)*** → ***assign guardrail initiatives in audit*** → ***tighten RBAC to least privilege*** → ***apply the secure-config baseline*** → ***enable Defender for Cloud and remediate top findings*** → ***validate on a pilot scope*** → ***move guardrails to deny*** → ***expand.*** *repeatable pattern:* ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand****, with an* ***audit-before-deny gate*** *and a documented rollback.*

2. Requirements Guide — Azure Security Baseline & Governance

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-azure-solutions.md`. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim.

2.1 An Azure subscription and tenant

Description: At least one Azure subscription under a Microsoft Entra tenant. **Why:** Policy, RBAC, and Defender for Cloud all operate against subscriptions anchored to a tenant. **Verify:** `portal.azure.com` → **Subscriptions**; run `AZSEC-Discovery.ps1`. **Implement-if-missing:** Create or associate a subscription under the tenant.

2.2 Elevated rights to assign policy and RBAC

Description: An operating account with **Owner** or **User Access Administrator** plus **Resource Policy Contributor** at the target scope (management group or subscription). **Why:** Assigning initiatives and creating role assignments requires these rights; Defender plan changes require **Security Admin** or Owner. **Verify:** Azure → **Access control (IAM)** at the target scope confirms the roles; `AZSEC-Discovery.ps1` reports current RBAC. **Implement-if-missing:** Assign the minimum roles at the narrowest scope; prefer PIM eligible activation over standing access.

2.3 A governance scope design (management groups / subscriptions)

Description: A decision on which **management group(s)** and **subscription(s)** the baseline applies to, and the pilot subset. **Why:** Assigning at the right node lets one guardrail cover many subscriptions; the pilot subset limits blast radius. **Verify:** `portal.azure.com` → **Management groups**; run `AZSEC-Discovery.ps1` (lists the hierarchy). **Implement-if-missing:** Create a management-group hierarchy and place subscriptions; use `AZSEC-Set-ScopeTargeting.ps1` to resolve and confirm the target scope.

2.4 Azure Policy available

Description: Azure Policy (built-in, no extra license) and the **Microsoft.PolicyInsights** resource provider registered. **Why:** Guardrail initiatives and compliance state require the provider registered on each subscription. **Verify:** Azure → **Subscriptions** → **Resource providers** shows **Microsoft.PolicyInsights** as Registered; `AZSEC-Validation.ps1` checks this. **Implement-if-missing:** Register the provider (`Register-AzResourceProvider -ProviderNamespace Microsoft.PolicyInsights`).

2.5 A secure-configuration baseline definition

Description: The agreed baseline: require encryption, restrict public storage/network access, allowed locations, allowed SKUs/resource types, secure transfer, diagnostic logging. **Why:** The baseline is what the guardrail initiatives enforce; without it there is nothing to assign. **Verify:** A written baseline exists and maps to specific built-in policies/initiatives. **Implement-if-missing:** Adopt the Microsoft cloud security benchmark initiative plus the named guardrail policies as the starting baseline; adjust to org standards.

2.6 Microsoft Defender for Cloud plan decision

Description: A decision on Defender for Cloud coverage — foundational CSPM (free, secure score + recommendations) and any enhanced/paid plans. **Why:** Secure score, regulatory compliance, and recommendations come from Defender for Cloud; enhanced plans add workload protections at a cost. **Verify:** `portal.azure.com` → **Microsoft Defender for Cloud** → **Environment settings**; `AZSEC-Discovery.ps1` reports plan status. **Implement-if-missing:** Enable foundational CSPM on in-scope subscriptions; enable enhanced plans where the risk justifies the cost.

2.7 Change control, pilot scope, and rollback

Description: An approved change window, a named pilot scope, and a documented rollback before moving any guardrail from audit to deny or tightening RBAC. **Why:** Deny policies and RBAC changes can block legitimate work; the audit-before-deny gate and rollback are the safety net. **Verify:** An approved change record, the pilot scope, and a written back-out exist (per `global-requirements-azure-solutions.md` G5/G6). **Implement-if-missing:** Export current

assignments (`AZSEC-Report.ps1`), define the pilot scope, and write the rollback (remove/reassign the initiative in audit).

Checklist

- Azure subscription + Entra tenant — `AZSEC-Discovery.ps1`
- Owner / User Access Administrator + Resource Policy Contributor at scope
- Governance scope design (management groups / subscriptions + pilot subset)
- Microsoft.PolicyInsights registered — `AZSEC-Validation.ps1`
- Secure-configuration baseline defined and mapped to policies
- Defender for Cloud plan decision made
- Change window, pilot scope, and rollback documented

3. Implementation Guide — Azure Security Baseline & Governance

Follow these steps **in order**, in the **Azure portal** (`https://portal.azure.com`). Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Assign guardrails in **audit** first, understand the picture, then move the **same** assignment to **deny** on a controlled gate. Tighten RBAC and apply the baseline on a **pilot scope**, validate, then expand. Keep a rollback for every deny/RBAC change.*

Naming (set once, reuse verbatim)

- Guardrail initiative assignment (audit stage): `Security Baseline Guardrails – Pilot MG – Audit`
- Guardrail initiative assignment (deny stage): `Security Baseline Guardrails – Pilot MG – Deny`
- Custom least-privilege role: `Resource Operator LeastPriv – Pilot Sub – Prod`
- Scope group (management group node): `Governance Scope – Pilot MG – Prod`

Step 1 — Define and confirm the scope

1. `portal.azure.com` → **Management groups**. Confirm (or create) the node that will hold the pilot subscriptions; treat it as `Governance Scope – Pilot MG – Prod`.
2. Place the pilot subscription(s) under that node.
3. Run `AZSEC-Set-ScopeTargeting.ps1` to resolve the management-group/subscription scope string used by every later step, and `AZSEC-Discovery.ps1` for the current policy/RBAC/Defender baseline.

Step 2 — Assign the guardrail initiative in audit

1. `portal.azure.com` → **Policy** → **Definitions** and pick the initiative to assign (start with the built-in Microsoft cloud security benchmark plus the named guardrails: require encryption, restrict public storage/network access, allowed locations, allowed SKUs).

2. **Assign**. **Scope:** the management-group node from Step 1 (`Governance Scope – Pilot MG – Prod`).

3. **Assignment name:** `Security Baseline Guardrails – Pilot MG – Audit`.

4. **Parameters:** set every guardrail **effect** to **Audit** (or **AuditIfNotExists**). Set allowed **locations** and allowed **SKUs** to your standards.

5. **Review + create.** No resource is blocked at this stage.

Step 3 — Review the audit picture and exempt false positives

1. **Policy** → **Compliance** → open `Security Baseline Guardrails – Pilot MG – Audit`.

2. Review non-compliant resources. For any legitimate exception, create a scoped **Exemption** (Policy → Exemptions) with a justification and expiry — do not weaken the policy.

3. Run `AZSEC-Report.ps1` to capture the audit-stage posture as the pre-change baseline.

Step 4 — Tighten RBAC to least privilege

1. `portal.azure.com` → the pilot subscription → **Access control (IAM)** → **Role assignments**. Identify **standing Owner** assignments that are broader than needed.

2. **Access control (IAM)** → **Add** → **Add custom role** → create `Resource Operator LeastPriv – Pilot Sub – Prod` granting only the actions operators actually need (deny sensitive `Microsoft.Authorization/*` write, etc.).

3. Reassign affected principals from **Owner** to the custom role (or to a scoped built-in role) at the narrowest scope. Prefer **PIM eligible** activation over standing access where available.

4. Run `AZSEC-Validation.ps1` to confirm the least-privilege state and that no unexpected standing Owner remains.

Step 5 — Apply the secure-configuration baseline

1. Confirm the baseline policies are part of the assigned initiative: require storage encryption, **deny/audit public blob access**, require **secure transfer (HTTPS)**, restrict **public network access**, allowed **locations** and **SKUs**, and diagnostic-logging policies.

2. Where a baseline control is not yet covered, add the built-in policy to the initiative (or assign it alongside) at the same scope, **in audit** for now.

3. Run `AZSEC-Report.ps1` to confirm the baseline controls appear in the posture report.

Step 6 — Enable Defender for Cloud and remediate top findings

1. `portal.azure.com` → **Microsoft Defender for Cloud** → **Environment settings** → the pilot subscription → enable **foundational CSPM** (and any enhanced plans agreed in requirements).

2. **Microsoft Defender for Cloud** → **Secure score** — record the starting score.

3. **Recommendations** — work the highest-impact items first; use **Fix** where offered.

4. **Regulatory compliance** — map to the standard(s) you must meet and note the gaps.

5. Run `AZSEC-Report.ps1` to capture secure score, top recommendations, and drift versus the baseline.

Step 7 — Validate on the pilot

1. Run `AZSEC-Validation.ps1` → confirms the audit-stage assignment is present, RBAC is least-privilege, and the provider is registered.

2. Run `AZSEC-Report.ps1` → confirms posture, secure score, and drift.

3. Attempt to create a resource that violates a guardrail (e.g. a storage account with public access) — in audit it is **allowed but flagged**. Complete the **Testing Guide** before proceeding.

Step 8 — Move guardrails from audit to deny (the gate)

1. **Gate:** only proceed when the audit picture is clean (exemptions in place, no surprise blocks expected) and the change is approved.

2. **Policy** → **Assignments** → duplicate the assignment as `Security Baseline Guardrails – Pilot MG – Deny`, or edit the existing one, setting each guardrail **effect** to **Deny**.

3. Re-run `AZSEC-Validation.ps1` and re-attempt the violating-resource creation — it must now be **blocked**.

4. **Rollback:** if deny blocks legitimate work, set the effect back to **Audit** (or remove the deny assignment) — resources already created are unaffected.

Step 9 — Expand to production

1. Reassign the validated initiative (in the effect stage you've proven) to the broader management-group node covering the remaining subscriptions.

2. Roll the custom role and RBAC tightening out to those subscriptions.

3. Enable Defender for Cloud on the expanded scope and re-baseline the secure score.

4. Re-validate each cycle with `AZSEC-Validation.ps1` and track drift with `AZSEC-Report.ps1`.

Order of operations

`Define scope → assign initiative (audit) → review/exempt → tighten RBAC → apply baseline → enable Defender + remediate → validate → move to deny (gate) → expand`

5. Testing Guide — Azure Security Baseline & Governance

Run after the audit-stage assignment applies (Implementation Step 7) and again after moving to deny (Step 8) and after each expansion cycle. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names.

Test 1 — Guardrail initiative assigned (audit)

Do: `portal.azure.com` → **Policy** → **Assignments** → confirm `Security Baseline Guardrails – Pilot MG – Audit` at the pilot scope; or run `AZSEC-Validation.ps1`. **Expected:** The assignment is present at the management-group node with every guardrail effect set to **Audit**.

Test 2 — Compliance state populated

Do: Policy → **Compliance** → open the assignment (run a scan if needed); or run `AZSEC-Report.ps1`. **Expected:** A per-policy compliant/non-compliant breakdown appears — the audit picture is visible, nothing is blocked.

Test 3 — RBAC is least privilege

Do: Run `AZSEC-Validation.ps1`; or **Access control (IAM)** → **Role assignments** on the pilot subscription. **Expected:** The custom role `Resource Operator LeastPriv – Pilot Sub – Prod` exists and is assigned; standing **Owner** assignments are reduced to only those justified.

Test 4 — Secure-config baseline present

Do: Run `AZSEC-Report.ps1`; or inspect the initiative's included policies. **Expected:** Encryption-required, public-access-restricted, secure-transfer, allowed-locations, and allowed-SKU controls all appear in the baseline.

Test 5 — Defender for Cloud posture

Do: Microsoft Defender for Cloud → **Secure score** and **Recommendations**; or run `AZSEC-Report.ps1`. **Expected:** A secure score is reported for the pilot subscription and top recommendations are listed; regulatory-compliance mapping is visible.

Test 6 — Audit does not block (audit stage)

Do: Create a resource that violates a guardrail (e.g. a storage account with public blob access) in the pilot scope. **Expected:** Creation **succeeds** but the resource shows as **non-compliant** — audit reports, it does not block.

Test 7 — Deny blocks (after the gate)

Do: After Step 8, re-attempt the same violating-resource creation. **Expected:** Creation is **blocked** by `Security Baseline Guardrails – Pilot MG – Deny`; a compliant resource still creates successfully.

Test 8 — Drift report

Do: Run `AZSEC-Report.ps1`. **Expected:** A clear governance-posture report — assignments, per-policy compliance, secure score, top recommendations, and drift versus the captured baseline.

Pass/fail gate

Gate	Requirement
Audit pass (pre-deny)	Tests 1–6 pass; the audit picture is understood and exemptions are in place
Deny pass	Test 7 passes — a violating resource is blocked and a compliant one still creates
Expand pass	Tests 1–5, 8 pass on the broader scope before the next expansion

6. Troubleshooting Guide — Azure Security Baseline & Governance

Use `AZSEC-Validation.ps1` / `AZSEC-Discovery.ps1` for scope state and `AZSEC-Report.ps1` for posture and drift, `AZSEC-Remediation.ps1` for safe fixes. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names.

A. Policy assignment not taking effect

Symptom	Cause	Remediation
Compliance shows blank / no resources	Evaluation hasn't run yet	Trigger a compliance scan (`Start-AzPolicyComplianceScan`); allow time — new assignments evaluate over minutes to hours
Assignment missing at scope	Assigned at the wrong node, or insufficient rights	Confirm scope with `AZSEC-Set-ScopeTargeting.ps1`; reassign at the management-group node; confirm Resource Policy Contributor
Provider errors	`Microsoft.PolicyInsights` not registered	`Register-AzResourceProvider -ProviderNamespace Microsoft.PolicyInsights`; re-run `AZSEC-Validation.ps1`

B. Deny blocks legitimate work

- This is why the model audits first. Create a scoped, justified, time-boxed **Exemption** for the specific resource — do not weaken the policy for everyone.

• If the block is broad or urgent, **roll back:** set the guardrail effect back to **Audit** (or remove the deny assignment). Resources already created are unaffected.

C. Deny didn't clean up existing resources

• Expected — deny only blocks *new* non-compliant resources. Remediate pre-existing ones with a **remediation task** (for DeployIfNotExists/Modify policies) or `AZSEC-Remediation.ps1`; audit-only findings must be fixed at the resource.

D. RBAC change broke automation or access

Symptom	Cause	Remediation
A service principal or user lost access	Standing Owner was removed too broadly	Reassign the custom role `Resource Operator LeastPriv – Pilot Sub – Prod` (or a scoped built-in role) at the needed scope; prefer PIM eligible
Cannot assign roles	Missing User Access Administrator/Owner	Grant the operating account the role at the target scope; re-run `AZSEC-Validation.ps1`

E. Defender for Cloud secure score not showing / not improving

• Confirm the plan is enabled on the subscription (**Environment settings**); foundational CSPM must be on for secure score and recommendations.

• Score updates on Defender's own cadence — allow time after remediating; re-run `AZSEC-Report.ps1` to capture the new score and remaining top recommendations.

F. Diagnostics quick reference

```
Get-AzContext | Select-Object Subscription, Tenant
Get-AzPolicyAssignment -Scope '<scope>'                       # assignments at a scope
Get-AzPolicyState -Filter "ComplianceState eq 'NonCompliant'" # non-compliant resources
Get-AzRoleAssignment -Scope '<scope>' | Where-Object RoleDefinitionName -eq 'Owner'  # standing Owner
Register-AzResourceProvider -ProviderNamespace Microsoft.PolicyInsights  # if unregistered
```

G. When to escalate

If the assignment is present at the correct scope, the provider is registered, a compliance scan has run, and compliance state or Defender secure score still does not populate after the normal evaluation window, collect `AZSEC-Report.ps1` output and the assignment/scope details and open a Microsoft support case.

7. Best Practices — Azure Security Baseline & Governance

• **Audit before you deny, always.** Assign every guardrail in audit first, understand the picture, exempt the legitimate exceptions, then flip the same assignment to deny on an approved gate. This one practice prevents the outage that comes from denying a config real workloads depend on.

• **Assign high in the hierarchy.** Put initiatives and role assignments at the management-group node, not per-subscription — one assignment then covers many subscriptions and new subscriptions inherit the baseline automatically.

• **Name everything and reuse the name.** Use `<Purpose> – <Scope> – <Stage>` for every assignment, custom role, and scope group, and reuse it verbatim across scripts, tests, and rollback so the estate stays auditable.

• **Least privilege by default.** Replace standing Owner with purpose-built custom roles at the narrowest scope; prefer PIM eligible activation over standing access. Owner is for break-glass, not day-to-day.

• **Deny stops new, not existing.** Remember a deny policy only blocks *new* non-compliant resources — remediate pre-existing ones separately with remediation tasks or the remediation script; don't assume deny cleaned up the estate.

• **Exempt, don't weaken.** For a genuine exception, create a scoped, justified, time-boxed exemption — never loosen the policy for everyone to satisfy one resource.

• **Track the secure score every cycle.** Record Defender for Cloud secure score at each stage and chase the top recommendations; a rising score is the evidence the baseline is working.

• **Keep a rollback for every deny/RBAC change.** Setting an effect back to audit or restoring a role assignment is the safety net; write it down before you make the change.

• **Right-size Defender plans.** Foundational CSPM (free) gives secure score and recommendations; turn on enhanced/paid plans only where the workload risk justifies the per-resource cost.

• **Pilot, then expand.** Prove the baseline on a small scope before rolling it estate-wide — the same assign-once, repeat-everywhere pattern that makes governance repeatable.

8. Deliverables Summary — Azure Security Baseline & Governance

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What Azure governance is — management groups, Azure Policy initiatives, RBAC least-privilege, secure-config baseline, Defender for Cloud; benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Azure Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal steps (scope, initiative audit→deny, RBAC, baseline, Defender)
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation

Document	Purpose
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`AZSEC-Discovery.ps1`	Reports policy assignments, RBAC (incl. standing Owner), and Defender for Cloud posture across the scope
`AZSEC-Validation.ps1`	Confirms the guardrail assignment, least-privilege RBAC, and provider registration are in place
`AZSEC-Report.ps1`	Governance-posture report — assignments, per-policy compliance, secure score, top recommendations, and drift
`AZSEC-Remediation.ps1`	Safe remediation (defaults to -WhatIf) — tightens RBAC and moves/assigns guardrail policy
`AZSEC-Set-ScopeTargeting.ps1`	Resolves and confirms the management-group / subscription scope used by every step

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the repeatable model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with an audit-before-deny gate and a documented rollback.

Outcome

Azure under governance — policy guardrails preventing misconfiguration, RBAC at least privilege, a secure-config baseline applied, and Defender for Cloud posture improving — proven on a pilot scope before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AZSEC-Discovery.ps1

Purpose: Read-only discovery for the "Azure Security Baseline & Governance" solution. For the scope in context (or a named subscription / management group), lists the Azure Policy assignments and their effect, the RBAC role assignments — flagging standing Owner — and the Defender for Cloud pricing/plan status and secure score. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AZSEC-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\AZSEC-Discovery.ps1 -SubscriptionId '<sub id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Discovery.ps1 -ManagementGroupId '<mg id>' Requires the Az PowerShell modules (Az.Accounts, Az.Resources, Az.PolicyInsights, Az.Security). Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A per-scope snapshot: policy assignments with effect, RBAC assignments with standing Owner highlighted, and Defender for Cloud plan status plus current secure score — a single readiness picture before you assign guardrails and tighten access. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. Reports on configuration/identifiers only; makes no changes to policy, RBAC, or Defender. #> [CmdletBinding()] param([string]$SubscriptionId, [string]$ManagementGroupId)

How to run: powershell -ExecutionPolicy Bypass -File .\AZSEC-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\AZSEC-Discovery.ps1 -SubscriptionId '<sub id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Discovery.ps1 -ManagementGroupId '<mg id>' Requires the Az PowerShell modules (Az.Accounts, Az.Resources, Az.PolicyInsights, Az.Security). Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A per-scope snapshot: policy assignments with effect, RBAC assignments with standing Owner highlighted, and Defender for Cloud plan status plus current secure score — a single readiness picture before you assign guardrails and tighten access. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. Reports on configuration/identifiers only; makes no changes to policy, RBAC, or Defender. #> [CmdletBinding()] param([string]$SubscriptionId, [string]$ManagementGroupId)

AZSEC-Remediation.ps1

Purpose: Safe, targeted fixes for the "Azure Security Baseline & Governance" solution. Two actions: 'TightenRbac' — remove a standing Owner role assignment for a principal at a scope, after (optionally) granting the least-privilege custom role 'Resource Operator LeastPriv - Pilot Sub - Prod' at the same scope; 'SetPolicyEffect' — set the 'effect' parameter of a named guardrail assignment to Audit or Deny (the audit-before-deny gate / rollback). DEFAULTS TO -WhatIf — it prints the change it would make and changes nothing until you pass -Execute. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\AZSEC-Remediation.ps1 -Action TightenRbac -PrincipalObjectId '<obj id>' -Scope '/subscriptions/<sub id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Remediation.ps1 -Action SetPolicyEffect -AssignmentName 'Security Baseline Guardrails - Pilot MG - Audit' -Effect Deny -Scope '/providers/Microsoft.Management/managementGroups/<mg id>' Apply for real (explicit): ... -Execute Requires Az.Accounts, Az.Resources. Run Connect-AzAccount first, or the script will prompt. TightenRbac requires Owner / User Access Administrator; SetPolicyEffect requires Resource Policy Contributor at the scope. EXPECTED RESULTS In -WhatIf mode (default): a printed description of the RBAC removal (+ optional grant) or the policy-effect change — nothing changes. With -Execute: the change is applied. .NOTES Workplace Marketplace — Azure Solutions. Defaults to -WhatIf. The

repeatable way. TightenRbac removes only the Owner assignment you name for the principal at the scope; it does not touch other assignments. SetPolicyEffect edits only the named assignment's effect parameter. #> [CmdletBinding()] param([Parameter(Mandatory)] [ValidateSet('TightenRbac','SetPolicyEffect')] [string]$Action, [Parameter(Mandatory)] [string]$Scope, [string]$PrincipalObjectId, # for TightenRbac [string]$LeastPrivRoleName = 'Resource Operator LeastPriv - Pilot Sub - Prod', # granted before removing Owner [switch]$GrantLeastPrivFirst, # for TightenRbac [string]$AssignmentName, # for SetPolicyEffect [ValidateSet('Audit','Deny')] [string]$Effect, # for SetPolicyEffect [string]$SubscriptionId, [switch]$Execute # omit for -WhatIf (default safe mode))

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\AZSEC-Remediation.ps1 -Action TightenRbac -PrincipalObjectId '<obj id>' -Scope '/subscriptions/<sub id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Remediation.ps1 -Action SetPolicyEffect -AssignmentName 'Security Baseline Guardrails - Pilot MG - Audit' -Effect Deny -Scope '/providers/Microsoft.Management/managementGroups/<mg id>' Apply for real (explicit): ... -Execute Requires Az.Accounts, Az.Resources. Run Connect-AzAccount first, or the script will prompt. TightenRbac requires Owner / User Access Administrator; SetPolicyEffect requires Resource Policy Contributor at the scope. EXPECTED RESULTS In -WhatIf mode (default): a printed description of the RBAC removal (+ optional grant) or the policy-effect change — nothing changes. With -Execute: the change is applied. .NOTES Workplace Marketplace — Azure Solutions. Defaults to -WhatIf. The repeatable way. TightenRbac removes only the Owner assignment you name for the principal at the scope; it does not touch other assignments. SetPolicyEffect edits only the named assignment's effect parameter. #> [CmdletBinding()] param([Parameter(Mandatory)] [ValidateSet('TightenRbac','SetPolicyEffect')] [string]$Action, [Parameter(Mandatory)] [string]$Scope, [string]$PrincipalObjectId, # for TightenRbac [string]$LeastPrivRoleName = 'Resource Operator LeastPriv - Pilot Sub - Prod', # granted before removing Owner [switch]$GrantLeastPrivFirst, # for TightenRbac [string]$AssignmentName, # for SetPolicyEffect [ValidateSet('Audit','Deny')] [string]$Effect, # for SetPolicyEffect [string]$SubscriptionId, [switch]$Execute # omit for -WhatIf (default safe mode))

AZSEC-Report.ps1

Purpose: Read-only reporting for the "Azure Security Baseline & Governance" solution. Produces a posture snapshot for the scope in context (or a named subscription): the policy assignments, a compliant/non-compliant summary from Azure Policy state, the Defender for Cloud secure score, and the top open recommendations by impact. If -BaselinePath points to a JSON file written by a previous run, reports the DRIFT (new non-compliant resources / score change). Optionally writes this run's summary to -BaselinePath for next-cycle drift comparison. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AZSEC-Report.ps1 powershell -ExecutionPolicy Bypass -File .\AZSEC-Report.ps1 -SubscriptionId '<sub id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Report.ps1 -BaselinePath .\azsec-baseline.json -SaveBaseline Requires Az.Accounts, Az.Resources, Az.PolicyInsights, Az.Security. Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A governance-posture report — assignments, non-compliant resource counts per policy, secure score, and top recommendations. With -BaselinePath, a drift line comparing this run to the saved baseline (score delta and new non-compliant count). .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. #> [CmdletBinding()] param([string]$SubscriptionId, [string]$BaselinePath, [switch]$SaveBaseline, [int]$TopRecommendations = 10)

How to run: powershell -ExecutionPolicy Bypass -File .\AZSEC-Report.ps1 powershell -ExecutionPolicy Bypass -File .\AZSEC-Report.ps1 -SubscriptionId '<sub id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Report.ps1 -BaselinePath .\azsec-baseline.json -SaveBaseline Requires Az.Accounts,

Az.Resources, Az.PolicyInsights, Az.Security. Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A governance-posture report — assignments, non-compliant resource counts per policy, secure score, and top recommendations. With -BaselinePath, a drift line comparing this run to the saved baseline (score delta and new non-compliant count). .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. #> [CmdletBinding()] param([string]$SubscriptionId, [string]$BaselinePath, [switch]$SaveBaseline, [int]$TopRecommendations = 10)

AZSEC-Set-ScopeTargeting.ps1

Purpose: Scope helper for the "Azure Security Baseline & Governance" solution. Resolves a target scope from either a management-group id or a subscription id, prints the resource-manager scope string that policy/RBAC assignments use, and lists the subscriptions that fall under it (for a management group, its child subscriptions; for a subscription, itself). Read-only — it confirms WHERE guardrails and roles will apply before you assign them. Use the printed scope string as the -Scope input to the other AZSEC scripts and the portal assignments. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AZSEC-Set-ScopeTargeting.ps1 -ManagementGroupId '<mg id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Set-ScopeTargeting.ps1 -SubscriptionId '<sub id>' Requires Az.Accounts, Az.Resources (management-group listing uses Get-AzManagementGroup from Az.Resources). Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS The resolved scope string (e.g. '/providers/Microsoft.Management/managementGroups/<id>' or '/subscriptions/<id>') and the list of subscriptions in scope — the confirmed target you reuse verbatim in every assignment, script, and test. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. Object naming convention: <Purpose> - <Scope> - <Stage>, e.g. 'Governance Scope - Pilot MG - Prod'. Reuse the resolved scope verbatim. #> [CmdletBinding()] param([string]$ManagementGroupId, [string]$SubscriptionId)

How to run: powershell -ExecutionPolicy Bypass -File .\AZSEC-Set-ScopeTargeting.ps1 -ManagementGroupId '<mg id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Set-ScopeTargeting.ps1 -SubscriptionId '<sub id>' Requires Az.Accounts, Az.Resources (management-group listing uses Get-AzManagementGroup from Az.Resources). Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS The resolved scope string (e.g. '/providers/Microsoft.Management/managementGroups/<id>' or '/subscriptions/<id>') and the list of subscriptions in scope — the confirmed target you reuse verbatim in every assignment, script, and test. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. Object naming convention: <Purpose> - <Scope> - <Stage>, e.g. 'Governance Scope - Pilot MG - Prod'. Reuse the resolved scope verbatim. #> [CmdletBinding()] param([string]$ManagementGroupId, [string]$SubscriptionId)

AZSEC-Validation.ps1

Purpose: Read-only validation for the "Azure Security Baseline & Governance" solution. Checks three things at the scope in context (or a named subscription / management group): 1. the named guardrail initiative assignment is present (default name 'Security Baseline Guardrails - Pilot MG - Audit', override with -AssignmentName); 2. RBAC is least-privilege — reports the count of standing Owner assignments and whether the custom role 'Resource Operator LeastPriv - Pilot Sub - Prod' exists; 3. the Microsoft.PolicyInsights resource provider is registered. Makes no changes; prints PASS/WARN per check. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AZSEC-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\AZSEC-Validation.ps1 -SubscriptionId '<sub id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Validation.ps1 -ManagementGroupId '<mg id>' -AssignmentName

'Security Baseline Guardrails - Pilot MG - Deny' Requires Az.Accounts, Az.Resources, Az.PolicyInsights. Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A short PASS/WARN report: guardrail assignment present, standing-Owner count (0 is best), custom least-privilege role present, and PolicyInsights registered — the go/no-go gate before moving guardrails to deny or expanding scope. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. #> [CmdletBinding()] param([string]$SubscriptionId, [string]$ManagementGroupId, [string]$AssignmentName = 'Security Baseline Guardrails - Pilot MG - Audit', [string]$CustomRoleName = 'Resource Operator LeastPriv - Pilot Sub - Prod')

How to run: powershell -ExecutionPolicy Bypass -File .\AZSEC-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\AZSEC-Validation.ps1 -SubscriptionId '<sub id>' powershell -ExecutionPolicy Bypass -File .\AZSEC-Validation.ps1 -ManagementGroupId '<mg id>' -AssignmentName 'Security Baseline Guardrails - Pilot MG - Deny' Requires Az.Accounts, Az.Resources, Az.PolicyInsights. Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A short PASS/WARN report: guardrail assignment present, standing-Owner count (0 is best), custom least-privilege role present, and PolicyInsights registered — the go/no-go gate before moving guardrails to deny or expanding scope. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. #> [CmdletBinding()] param([string]$SubscriptionId, [string]$ManagementGroupId, [string]$AssignmentName = 'Security Baseline Guardrails - Pilot MG - Audit', [string]$CustomRoleName = 'Resource Operator LeastPriv - Pilot Sub - Prod')